RAPPORT DE TÂCHE

Mission 2 — Tâche 2 : Gérer les commandes de revues

Projet MediatekDocuments — C# / PHP / MySQL

Projet	MediatekDocuments — BTS SIO SLAM
Mission	Mission 2 — Gérer les commandes
Tâche	Tâche 2 — Gérer les commandes de revues (abonnements)
Temps estimé	4 heures
Temps réel	12 heures
Fichiers C# modifiés	FrmMediatek.cs, FrmMediatek.Designer.cs, FrmMediatekController.cs, Access.cs, CommandeRevue.cs (nouveau), FrmAlerteAbonnements.cs (nouveau)
Fichiers PHP modifiés	MyAccessBDD.php, .htaccess
BDD modifiée	Table abonnement (colonnes ajoutées), contrainte clé étrangère supprimée, taille id modifiée
Tests unitaires	5 tests — MediaTekDocuments.Tests — TestParutionDansAbonnement

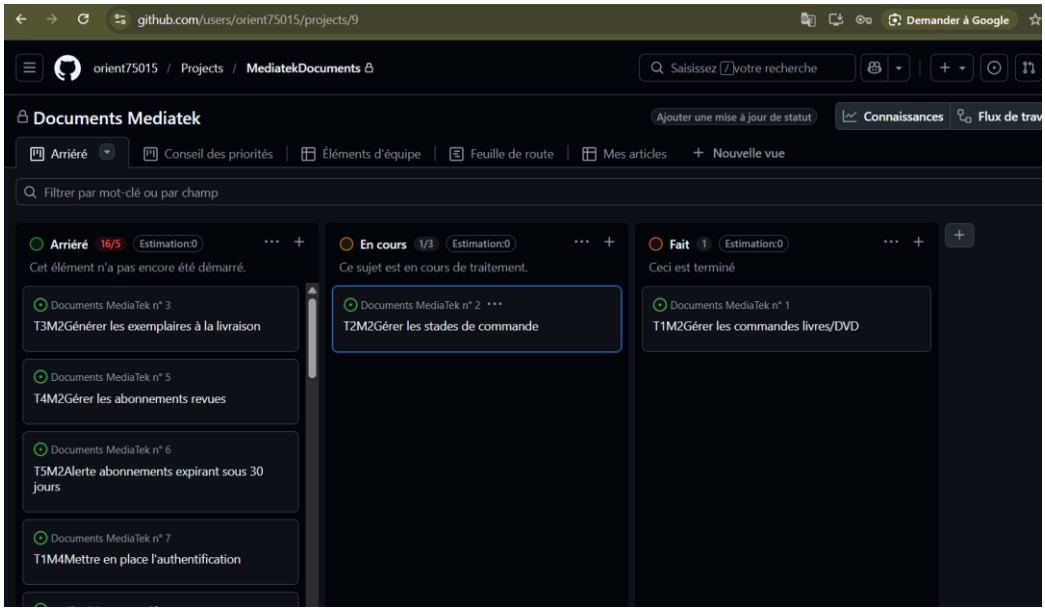
1. Demande de la tâche

La tâche 2 de la Mission 2 consiste à créer un onglet de gestion des commandes de revues (abonnements). Les fonctionnalités demandées sont :

- Créer un onglet pour gérer les abonnements (nouvel abonnement ou renouvellement)
- Respecter la charte graphique existante
- Permettre le tri sur toutes les colonnes
- Sélectionner une revue par numéro et afficher ses informations ainsi que la liste des abonnements triée par date (ordre inverse)
- Créer un groupbox pour saisir et enregistrer un nouvel abonnement
- Supprimer un abonnement uniquement si aucun exemplaire n'est rattaché (méthode ParutionDansAbonnement)
- Créer le test unitaire sur la méthode ParutionDansAbonnement
- Afficher une fenêtre d'alerte au démarrage pour les abonnements expirant dans moins de 30 jours

2. Kanban

Capture du Kanban — FrmMediatek.cs (onglet Git → Historique ou GitHub)



Capture d'écran du Kanban GitHub avec T2M2 dans la colonne 'In Progress'

3. Temps estimé et temps réel

Indicateur	Valeur
Temps estimé	4 heures
Temps réel	12 heures
Écart	+8 heures (nombreux problèmes techniques rencontrés — voir section 7)

4. Diagramme de cas d'utilisation

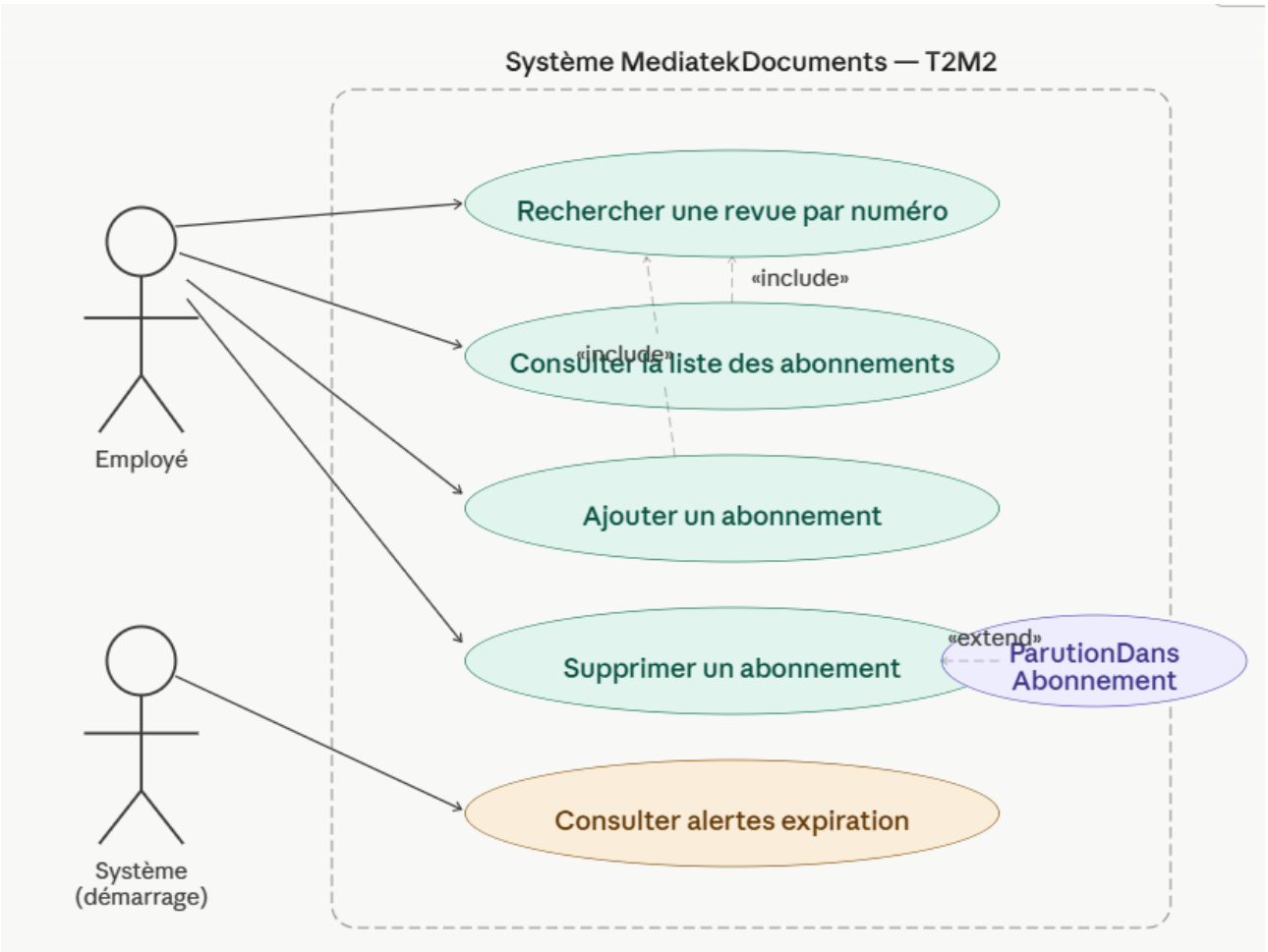
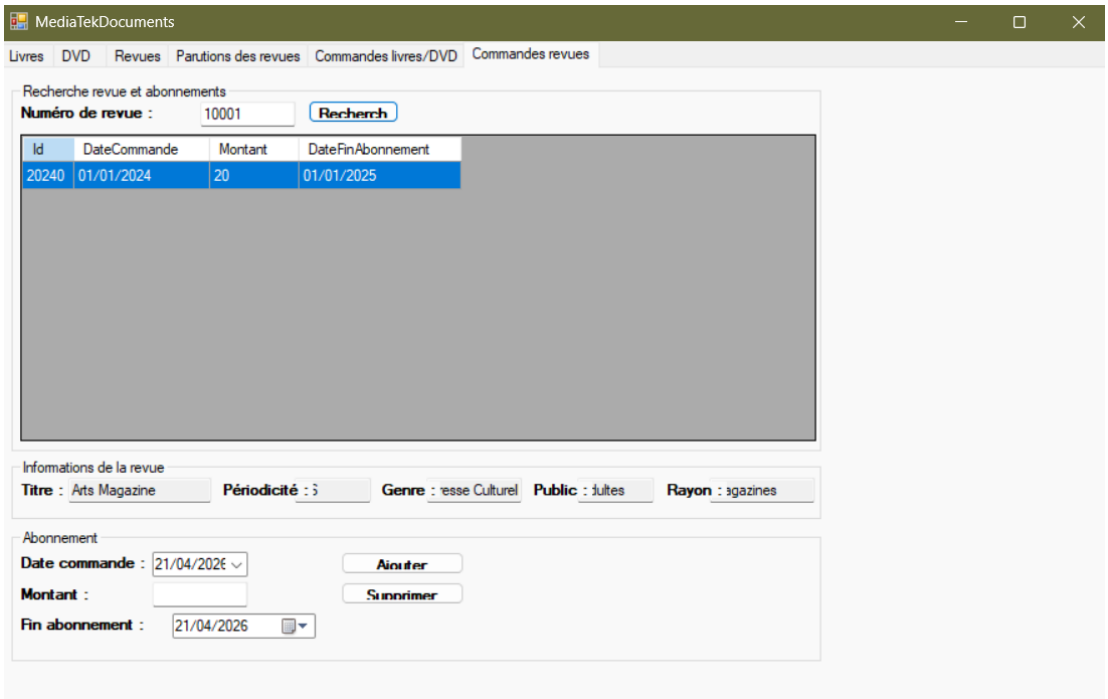


Diagramme de cas d'utilisation T2M2 (à insérer depuis Looping ou draw.io)

Acteur : Employé (service administratif). Cas d'utilisation : Rechercher une revue par numéro, Consulter la liste des abonnements, Ajouter un abonnement, Supprimer un abonnement, Consulter les alertes d'expiration.

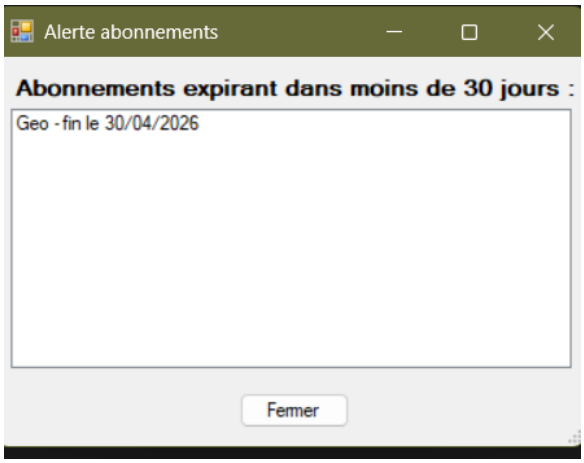
5. Maquette de la page



Capture de l'onglet 'Commandes revues' dans l'application — FrmMediatek.cs

L'onglet contient trois zones principales :

- Zone de recherche (haut) : saisie du numéro de revue + bouton Rechercher + datagrid des abonnements
- Zone informations revue (milieu) : affichage du titre, périodicité, genre, public, rayon
- Zone saisie abonnement (bas) : date de commande, montant, date de fin + boutons Ajouter et Supprimer



Capture de la fenêtre d'alerte FrmAlerteAbonnements.cs au démarrage

6. Classes impactées

6.1 CommandeRevue.cs (nouveau modèle)

Nouvelle classe métier représentant un abonnement à une revue.

```
public class CommandeRevue
  + Id : string
  + DateCommande : DateTime
  + Montant : double
  + DateFinAbonnement : DateTime
  + IdRevue : string
  + Titre : string
  + CommandeRevue(id, dateCommande, montant, dateFinAbonnement, idRevue)
```

CommandeRevue.cs | FrmMediatek.Designer.cs | FrmMediatek.cs

MediaTekDocuments | MediaTekDocuments.model.CommandeRevue

```
using System;

namespace MediaTekDocuments.model
{
    /// <summary>
    /// Classe métier CommandeRevue (abonnement ou renouvellement d'une revue)
    /// </summary>
    30 références | 0 modification | 0 auteur, 0 modification
    public class CommandeRevue
    {
        3 références | 0 modification | 0 auteur, 0 modification
        public string Id { get; set; }
        4 références | 0 modification | 0 auteur, 0 modification
        public DateTime DateCommande { get; set; }
        3 références | 0 modification | 0 auteur, 0 modification
        public double Montant { get; set; }
        5 références | 0 modification | 0 auteur, 0 modification
        public DateTime DateFinAbonnement { get; set; }
        3 références | 0 modification | 0 auteur, 0 modification
        public string IdRevue { get; set; }
        1 référence | 0 modification | 0 auteur, 0 modification
        public string Titre { get; set; }

        1 référence | 0 modification | 0 auteur, 0 modification
        public CommandeRevue(string id, DateTime dateCommande, double montant,
            DateTime dateFinAbonnement, string idRevue)
        {
            this.Id = id;
            this.DateCommande = dateCommande;
            this.Montant = montant;
            this.DateFinAbonnement = dateFinAbonnement;
            this.IdRevue = idRevue;
        }
    }
```

95 % | Aucun problème détecté

6.2 Access.cs (dal)

Ajout de 4 nouvelles méthodes d'accès à l'API REST :

```
+ GetCommandesRevue(idRevue : string) : List<CommandeRevue>
```

Récupère la liste des abonnements d'une revue triée par date décroissante.

```
+ CreerCommandeRevue(commande : CommandeRevue) : bool
```

Insère un nouvel abonnement dans les tables commande et abonnement via l'API.

```
+ SupprimerCommandeRevue(id : string) : bool
```

Supprime un abonnement (uniquement si aucun exemplaire rattaché, vérifié côté C#).

```
+ GetAbonnementsExpiresBientot() : List<CommandeRevue>
```

Retourne les abonnements expirant dans moins de 30 jours, triés par date croissante.

```
        }

        /// <summary>
        /// Retourne les revues dont l'abonnement se termine dans moins de 30 jours
        /// </summary>
        1 référence | 0 modification | 0 auteur, 0 modification
        public List<CommandeRevue> GetAbonnementsExpiresBientot()
        {
            List<CommandeRevue> lesAbonnements = TraitementRecup<CommandeRevue>(GET, "abonnement/expiresbientot", null);
            return lesAbonnements;
        }

    }
}
```

```
1 référence | 0 modification | 0 auteur, 0 modification
public List<CommandeRevue> GetCommandesRevue(string idRevue)
{
    String jsonId = convertToJson("id", idRevue);
    List<CommandeRevue> lesCommandes = TraitementRecup<CommandeRevue>(GET, "abonnement/" + jsonId, null);
    return lesCommandes;
}

/// <summary>
/// Crée une commande de revue (abonnement)
/// </summary>
1 référence | 0 modification | 0 auteur, 0 modification
public bool CreerCommandeRevue(CommandeRevue commande)
{
    String jsonCommande = JsonConvert.SerializeObject(commande, new CustomDateTimeConverter());
    try
    {
        List<CommandeRevue> liste = TraitementRecup<CommandeRevue>(POST, "abonnement", "champs=" + jsonCommande);
        return (liste != null);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
    return false;
}

/// <summary>
/// Supprime une commande de revue
/// </summary>
1 référence | 0 modification | 0 auteur, 0 modification
public bool SupprimerCommandeRevue(string id)
{
    String jsonId = convertToJson("id", id);
    try
    {
        List<CommandeRevue> liste = TraitementRecup<CommandeRevue>(DELETE, "abonnement/" + jsonId, null);
        return (liste != null);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
    return false;
}

/// <summary>
/// Retourne les revues dont l'abonnement se termine dans moins de 30 jours
/// </summary>
```

6.3 FrmMediatekController.cs (controller)

Ajout des 4 méthodes de délégation vers Access.cs :

```
+ GetCommandesRevue(idRevue : string) : List<CommandeRevue>
+ CreerCommandeRevue(commande : CommandeRevue) : bool
+ SupprimerCommandeRevue(id : string) : bool
+ GetAbonnementsExpiresBientot() : List<CommandeRevue>
```

```
3 références | 0 modification | 0 auteur, 0 modification
public List<CommandeRevue> GetCommandesRevue(string idRevue)
    {
        return access.GetCommandesRevue(idRevue);
    }

    1 référence | 0 modification | 0 auteur, 0 modification
    public bool CreerCommandeRevue(CommandeRevue commande)
    {
        return access.CreerCommandeRevue(commande);
    }

    1 référence | 0 modification | 0 auteur, 0 modification
    public bool SupprimerCommandeRevue(string id)
    {
        return access.SupprimerCommandeRevue(id);
    }

    1 référence | 0 modification | 0 auteur, 0 modification
    public List<CommandeRevue> GetAbonnementsExpiresBientot()
    {
        return access.GetAbonnementsExpiresBientot();
    }

}

}
```

6.4 FrmMediatek.cs (vue) — méthodes ajoutées

`+ tabCommandesRevues_Enter(sender, e) : void`

Initialise l'onglet : vide les champs et le datagrid.

`+ btnCommandesRevuesRechercher_Click(sender, e) : void`

Recherche la revue par numéro, affiche ses infos et la liste de ses abonnements.

`+ RemplirCommandesRevuesListe(commandes : List<CommandeRevue>) : void`

Remplit le datagrid avec la liste des abonnements, masque les colonnes inutiles.

`+ dgvCommandesRevuesListe_SelectionChanged(sender, e) : void`

Affiche les infos de l'abonnement sélectionné dans les champs du formulaire.

`+ btnCommandesRevuesAjouter_Click(sender, e) : void`

Valide les saisies (montant, date fin > date début) et crée un nouvel abonnement.

`+ btnCommandesRevuesSupprimer_Click(sender, e) : void`

Vérifie qu'aucun exemplaire n'est rattaché avant de supprimer l'abonnement.

`+ dgvCommandesRevuesListe_ColumnHeaderMouseClick(sender, e) : void`

Tri sur les colonnes Id, DateCommande, Montant, DateFinAbonnement.

`+ static ParutionDansAbonnement(dateCommande, dateFinAbonnement, dateParution) : bool`

Retourne vrai si dateParution est comprise entre dateCommande et dateFinAbonnement (inclus).

```
1432        /// Ouverture de l'onglet Commandes revues
1433        /// </summary>
            1 référence | 0 modification | 0 auteur, 0 modification
1434        private void tabCommandesRevues_Enter(object sender, EventArgs e)
1435        {
1436            lesCommandesRevues = new List<CommandeRevue>();
1437            dgvCommandesRevuesListe.DataSource = null;
1438            bdgCommandesRevuesListe.DataSource = lesCommandesRevues;
1439            txbCommandesRevuesTitre.Text = "";
1440            txbCommandesRevuesPeriodicite.Text = "";
1441            txbCommandesRevuesGenre.Text = "";
1442            txbCommandesRevuesPublic.Text = "";
1443            txbCommandesRevuesRayon.Text = "";
1444            this.Text = "MediaTekDocuments";
1445        }
1446
1447        /// <summary>
1448        /// Recherche des abonnements d'une revue
1449        /// </summary>
            1 référence | 0 modification | 0 auteur, 0 modification
1450        private void btnCommandesRevuesRechercher_Click(object sender, EventArgs e)
1451        {
1452            if (!txbCommandesRevuesNumero.Text.Equals(""))
1453            {
1454                List<Revue> lesRevues = controller.GetAllRevues();
1455                Revue revue = lesRevues.Find(x => x.Id.Equals(txbCommandesRevuesNumero.Text));
1456                if (revue == null)
1457                {
1458                    MessageBox.Show("Numéro de revue introuvable", "Information");
1459                    return;
1460                }
1461                txbCommandesRevuesTitre.Text = revue.Titre;
1462                txbCommandesRevuesPeriodicite.Text = revue.Periodicite;
1463                txbCommandesRevuesGenre.Text = revue.Genre;
1464                txbCommandesRevuesPublic.Text = revue.Public;
1465                txbCommandesRevuesRayon.Text = revue.Rayon;
1466                lesCommandesRevues = controller.GetCommandesRevue(txbCommandesRevuesNumero.Text);
1467                RemplirCommandesRevuesListe(lesCommandesRevues);
1468            }
1469            else
1470            {
1471                MessageBox.Show("Veuillez saisir un numéro de revue", "Information");
1472            }
1473        }
```

```
1476        /// Remplit le datagrid avec la liste des abonnements
1477        /// </summary>
            4 références | 0 modification | 0 auteur, 0 modification
1478        private void RemplirCommandesRevuesListe(List<CommandeRevue> commandes)
1479        {
1480            bdgCommandesRevuesListe.DataSource = commandes;
1481            dgvCommandesRevuesListe.DataSource = bdgCommandesRevuesListe;
1482            if (dgvCommandesRevuesListe.Columns.Count > 0)
1483            {
1484                dgvCommandesRevuesListe.Columns["idRevue"].Visible = false;
1485                dgvCommandesRevuesListe.Columns["titre"].Visible = false;
1486                dgvCommandesRevuesListe.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.AllCells;
1487            }
1488        }
```

```
1516        /// </summary>
            1 référence | 0 modification | 0 auteur, 0 modification
1517        private void btnCommandesRevuesAjouter_Click(object sender, EventArgs e)
1518        {
1519            if (txbCommandesRevuesNumero.Text.Equals(""))
1520            {
1521                MessageBox.Show("Veuillez saisir un numéro de revue", "Information");
1522                return;
1523            }
1524            if (txbCommandesRevuesMontant.Text.Equals(""))
1525            {
1526                MessageBox.Show("Veuillez remplir le montant", "Information");
1527                return;
1528            }
1529            if (dtpCommandesRevuesDateFin.Value <= dtpCommandesRevuesDate.Value)
1530            {
1531                MessageBox.Show("La date de fin doit être postérieure à la date de commande", "Information");
1532                return;
1533            }
1534            try
1535            {
1536                string id = DateTime.Now.ToString("yyyyMMddHHmmss");
1537                DateTime dateCommande = dtpCommandesRevuesDate.Value;
1538                double montant = double.Parse(txbCommandesRevuesMontant.Text);
1539                DateTime dateFinAbonnement = dtpCommandesRevuesDateFin.Value;
1540                string idRevue = txbCommandesRevuesNumero.Text;
1541                CommandeRevue commande = new CommandeRevue(id, dateCommande, montant, dateFinAbonnement, idRevue);
1542                if (controller.CreerCommandeRevue(commande))
1543                {
1544                    lesCommandesRevues = controller.GetCommandesRevue(idRevue);
1545                    RemplirCommandesRevuesListe(lesCommandesRevues);
1546                }
1547                else
1548                {
1549                    MessageBox.Show("Erreur lors de l'ajout", "Erreur");
1550                }
1551            }
1552            catch
1553            {
1554                MessageBox.Show("Veuillez vérifier les valeurs saisies", "Information");
1555            }
1556        }
```

```
            1 référence | 0 modification | 0 auteur, 0 modification
1561        private void btnCommandesRevuesSupprimer_Click(object sender, EventArgs e)
1562        {
1563            if (dgvCommandesRevuesListe.CurrentCell == null)
1564            {
1565                MessageBox.Show("Veuillez sélectionner un abonnement", "Information");
1566                return;
1567            }
1568            CommandeRevue commande = (CommandeRevue)bdgCommandesRevuesListe.List[bdgCommandesRevuesListe.Position];
1569            List<Exemplaire> exemplaires = controller.GetExemplairesRevue(commande.IdRevue);
1570            bool aDesExemplaires = exemplaires.Exists(ex =>
1571                ParutionDansAbonnement(commande.DateCommande, commande.DateFinAbonnement, ex.DateAchat));
1572            if (aDesExemplaires)
1573            {
1574                MessageBox.Show("Impossible de supprimer : des exemplaires sont rattachés à cet abonnement", "Infor
1575                return;
1576            }
1577            if (MessageBox.Show("Confirmer la suppression ?", "Confirmation", MessageBoxButtons.YesNo) == DialogRes
1578            {
1579                if (controller.SupprimerCommandeRevue(commande.Id))
1580                {
1581                    lesCommandesRevues = controller.GetCommandesRevue(commande.IdRevue);
1582                    RemplirCommandesRevuesListe(lesCommandesRevues);
1583                }
1584                else
1585                {
1586                    MessageBox.Show("Erreur lors de la suppression", "Erreur");
1587                }
1588            }
1589        }
```

```
1597        /// <returns>true si la parution est dans la période d'abonnement</returns>
            6 références | 5/5 passing | 0 modification | 0 auteur, 0 modification
1598        public static bool ParutionDansAbonnement(DateTime dateCommande, DateTime dateFinAbonnement, DateTime dateParution)
1599        {
1600            return dateParution >= dateCommande && dateParution <= dateFinAbonnement;
1601        }
1602
```

6.5 FrmAlerteAbonnements.cs (nouvelle vue)

Nouvelle fenêtre affichée au démarrage si des abonnements expirent dans moins de 30 jours.

```
+ FrmAlerteAbonnements(abonnements : List<CommandeRevue>)
```

Constructeur : remplit la liste avec titre et date de fin de chaque abonnement.

```
+ btnFermer_Click(sender, e) : void
```

Ferme la fenêtre d'alerte.

```
1   using System;
2   using System.Collections.Generic;
3   using System.Windows.Forms;
4   using MediaTekDocuments.model;
5
6   namespace MediaTekDocuments.view
7   {
        4 références  0 modification  0 auteur, 0 modification
8       public partial class FrmAlerteAbonnements : Form
9       {
            1 référence  0 modification  0 auteur, 0 modification
10          public FrmAlerteAbonnements(List<CommandeRevue> abonnements)
11          {
12              InitializeComponent();
13              foreach (CommandeRevue abonnement in abonnements)
14              {
15                  lstAbonnements.Items.Add(abonnement.Titre + " - fin le " +
16                      abonnement.DateFinAbonnement.ToString("dd/MM/yyyy"));
17              }
18          }
19
            1 référence  0 modification  0 auteur, 0 modification
20          private void btnFermer_Click(object sender, EventArgs e)
21          {
22              this.Close();
23          }
24      }
25  }
```

6.6 MyAccessBDD.php (API PHP)

Ajout de 4 fonctions et des cases correspondants dans traitementSelect, traitementInsert et traitementDelete :

```
- selectCommandesRevue(?array $champs) : ?array
```

Requête SELECT avec jointure commande + abonnement filtrée par idRevue.

```
- selectAbonnementsExpiresBientot() : ?array
```

Requête SELECT avec jointure commande + abonnement + document, filtrée sur les 30 prochains jours.

```
- insertCommandeRevue(?array $champs) : ?int
```

Double INSERT : dans commande puis dans abonnement.

```
- deleteCommandeRevue ou deleteTuplesOneTable
```

Suppression en cascade via clé étrangère supprimée (voir problèmes rencontrés).

```
329     }
330     private function selectAbonnementsExpiresBientot() : ?array{
331     $requete = "SELECT a.id, a.dateCommande, a.montant, a.dateFinAbonnement, a.idRevue, d.titre ";
332     $requete .= "FROM abonnement a ";
333     $requete .= "JOIN document d ON a.idRevue = d.id ";
334     $requete .= "WHERE a.dateFinAbonnement BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 30 DAY) ";
335     $requete .= "ORDER BY a.dateFinAbonnement ASC";
336     return $this->conn->queryBDD($requete);
337
338
```

7. Problèmes rencontrés

7.1 Structure de la table abonnement incorrecte

La table abonnement existante ne contenait que id, dateFinAbonnement et idRevue. Les colonnes dateCommande et montant manquaient.

Solution : ALTER TABLE abonnement ADD COLUMN dateCommande DATE NOT NULL et ADD COLUMN montant DOUBLE NOT NULL.

7.2 Contrainte de clé étrangère bloquant l'insertion

La table abonnement avait une contrainte FOREIGN KEY sur id référençant la table commande. Cela empêchait l'insertion directe dans abonnement.

Solution : ALTER TABLE abonnement DROP FOREIGN KEY abonnement_ibfk_1.

7.3 Taille du champ id trop petite

Le champ id de la table abonnement était VARCHAR(5), trop court pour les ids générés en C# au format yyyyMMddHHmmss (14 caractères).

Solution : ALTER TABLE abonnement MODIFY id VARCHAR(20) NOT NULL.

7.4 Route API abonnement/expiresbientot non reconnue

La règle de réécriture dans .htaccess ne prenait pas en charge les routes avec deux segments texte (ex: abonnement/expiresbientot). L'API retournait du HTML au lieu de JSON.

Solution : ajout d'une règle RewriteRule dans .htaccess pour les GET avec deux segments alphanumériques séparés par un slash.

7.5 Cases PHP placées après le default dans le switch

Les cases abonnement dans traitementInsert et traitementDelete étaient placées après le default, ce qui les rendait inatteignables.

Solution : repositionner les cases avant le default dans chaque switch.

7.6 Problème de licence Visual Studio

La licence Visual Studio 2026 Professional a expiré en cours de développement, bloquant la compilation.

Solution : reconnexion avec un compte Microsoft pour prolonger l'accès.

7.7 MySQL s'arrêtant de manière inattendue

Le service MySQL de XAMPP s'est arrêté spontanément pendant les tests, provoquant des erreurs 500 sur toutes les requêtes API.

Solution : redémarrage manuel de MySQL depuis le panneau XAMPP.

7.8 Double déclaration lblCommandesRevuesTitre dans le Designer

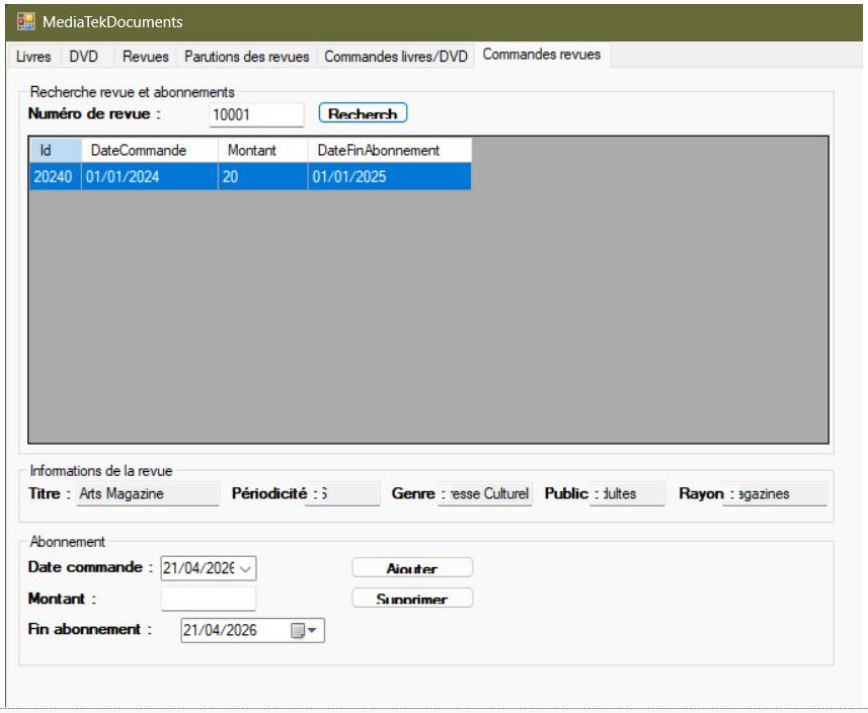
En ajoutant les nouveaux contrôles d'affichage des infos de la revue, le label lblCommandesRevuesTitre a été déclaré deux fois dans FrmMediatek.Designer.cs, provoquant une erreur CS0102.

Solution : suppression de la déclaration en double.

8. Résultat

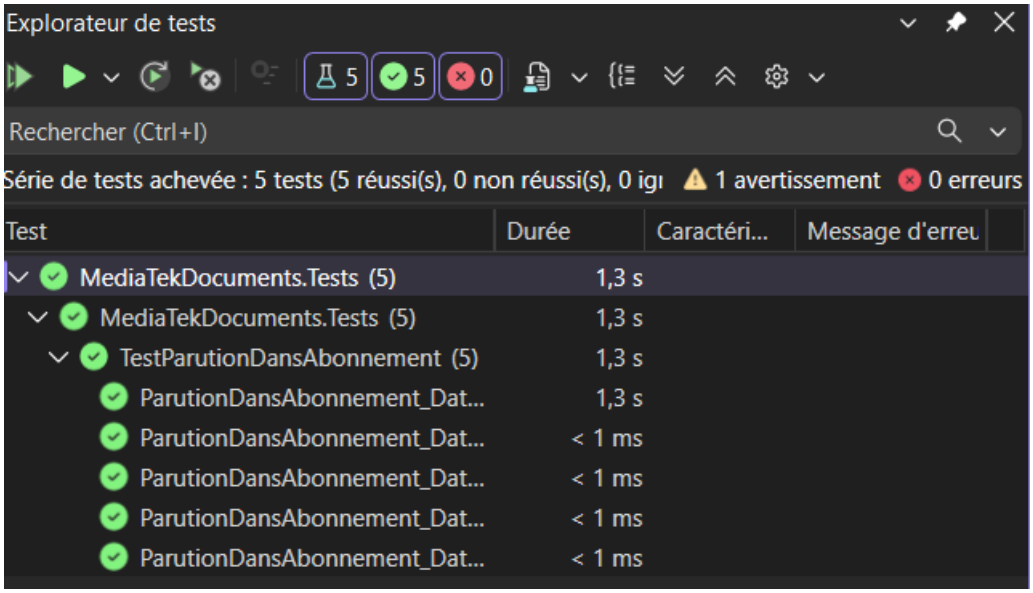
La gestion des commandes de revues est opérationnelle et respecte toutes les contraintes imposées :

- Onglet 'Commandes revues' fonctionnel avec charte graphique conforme
- Recherche par numéro de revue avec affichage des informations (titre, périodicité, genre, public, rayon)
- Liste des abonnements triée par date décroissante avec tri sur toutes les colonnes
- Ajout d'abonnement avec validation (montant obligatoire, date fin > date début)
- Suppression sécurisée via la méthode ParutionDansAbonnement
- 5 tests unitaires créés et réussis sur la méthode ParutionDansAbonnement
- Fenêtre d'alerte au démarrage pour les abonnements expirant dans moins de 30 jours



Capture de l'onglet 'Commandes revues' après recherche avec abonnements affichés

```
9   {
10      [TestMethod]
        0 références | 0 modification | 0 auteur, 0 modification
11      public void ParutionDansAbonnement_DateDansIntervalle_RetourneVrai()
12      {
13          DateTime dateCommande = new DateTime(2024, 1, 1);
14          DateTime dateFinAbonnement = new DateTime(2024, 12, 31);
15          DateTime dateParution = new DateTime(2024, 6, 15);
16          bool resultat = FrmMediatek.ParutionDansAbonnement(dateCommande, dateFinAbonnement, dateParution);
17          Assert.IsTrue(resultat);
18      }
19
20      [TestMethod]
        0 références | 0 modification | 0 auteur, 0 modification
21      public void ParutionDansAbonnement_DateAvantIntervalle_RetourneFaux()
22      {
23          DateTime dateCommande = new DateTime(2024, 1, 1);
24          DateTime dateFinAbonnement = new DateTime(2024, 12, 31);
25          DateTime dateParution = new DateTime(2023, 12, 31);
26          bool resultat = FrmMediatek.ParutionDansAbonnement(dateCommande, dateFinAbonnement, dateParution);
27          Assert.IsFalse(resultat);
28      }
29
30      [TestMethod]
        0 références | 0 modification | 0 auteur, 0 modification
31      public void ParutionDansAbonnement_DateApresIntervalle_RetourneFaux()
32      {
33          DateTime dateCommande = new DateTime(2024, 1, 1);
34          DateTime dateFinAbonnement = new DateTime(2024, 12, 31);
35          DateTime dateParution = new DateTime(2025, 1, 1);
36          bool resultat = FrmMediatek.ParutionDansAbonnement(dateCommande, dateFinAbonnement, dateParution);
37          Assert.IsFalse(resultat);
38      }
39
40      [TestMethod]
        0 références | 0 modification | 0 auteur, 0 modification
41      public void ParutionDansAbonnement_DateEgaleDebut_RetourneVrai()
42      {
43          DateTime dateCommande = new DateTime(2024, 1, 1);
44          DateTime dateFinAbonnement = new DateTime(2024, 12, 31);
45          DateTime dateParution = new DateTime(2024, 1, 1);
46          bool resultat = FrmMediatek.ParutionDansAbonnement(dateCommande, dateFinAbonnement, dateParution);
47          Assert.IsTrue(resultat);
48      }
49
50      [TestMethod]
        0 références | 0 modification | 0 auteur, 0 modification
51      public void ParutionDansAbonnement_DateEgaleFin_RetourneVrai()
52      {
53          DateTime dateCommande = new DateTime(2024, 1, 1);
54          DateTime dateFinAbonnement = new DateTime(2024, 12, 31);
55          DateTime dateParution = new DateTime(2024, 12, 31);
56          bool resultat = FrmMediatek.ParutionDansAbonnement(dateCommande, dateFinAbonnement, dateParution);
57          Assert.IsTrue(resultat);
```



Capture des 5 tests unitaires réussis — MediaTekDocuments.Tests